Project 2 - LLM Usage Report

LLM Usage Documentation

Purpose of this Report

This document records how **Large Language Models (LLMs)** were used during the development of features.

As required in the project instructions, we documented:

- the **prompts provided to the LLM**
- the **responses generated**
- a **critical evaluation of their usefulness and accuracy**

Screenshots are included as evidence of the prompt–response interactions. For each interaction we summarize:

- **How the response influenced the implementation**
- **What parts of the response were correct or useful**
- **What required manual correction or adaptation**
- **How the final implementation decisions were made**

The goal of this documentation is **not to show code generation**, but to demonstrate how LLM assistance was used as a **design and debugging aid** while maintaining full control over architectural and implementation decisions.

How LLM Assistance Was Used

Throughout the project, LLMs were primarily used for:

- **Requirement clarification and feature decomposition**
- **Architecture brainstorming and design validation**
- **Query and algorithm suggestions**
- **Debugging guidance and troubleshooting steps**
- **Testing strategies and edge-case identification**

In most cases, the LLM output served as a **high-level starting point** rather than a final solution. All suggestions were manually reviewed and adapted to ensure compatibility with the **existing Apache Roller architecture**, project constraints, and assignment requirements.

Accuracy Evaluation Approach

For every documented interaction we classify the output into three categories:

- **Correct / Useful** — Ideas or explanations that directly helped guide the implementation.
- **Incomplete / Adjusted** — Responses that contained useful concepts but required modification to fit the project codebase.
- **Net Evaluation** — A short summary of the overall usefulness of the response.

This evaluation ensures transparency regarding **how much of the LLM output was actually used** and highlights where **manual engineering decisions were necessary**.

Task 1A

Purpose

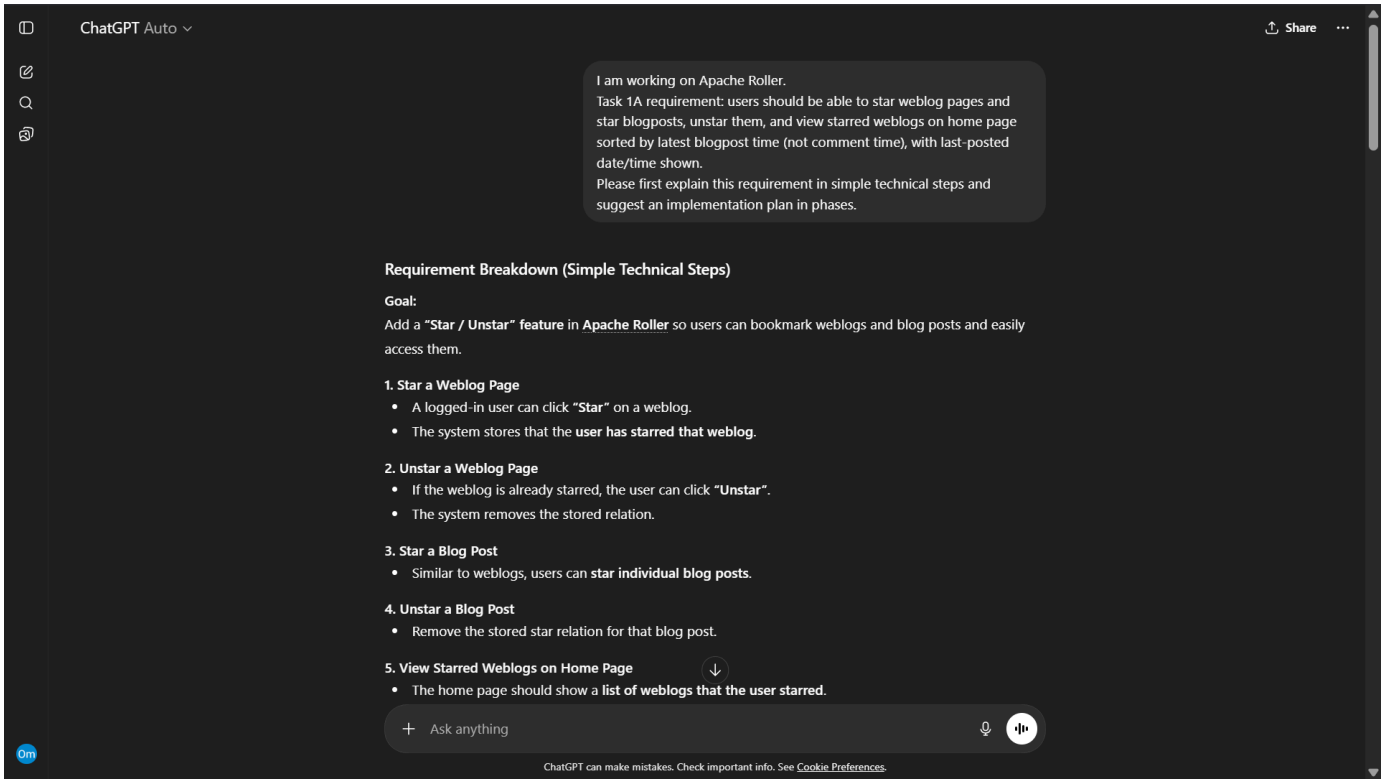
This document records how LLM assistance was used while implementing **Task 1A (Stars)**.

The screenshots below capture the prompt/response workflow used during coding, and the evaluation notes summarize what was correct, what required fixes, and what was finally adopted.

Screenshot Evidence

Note: This evidence is from one continuous prompt-response flow captured across six screenshots.

1) Initial prompt and requirement breakdown



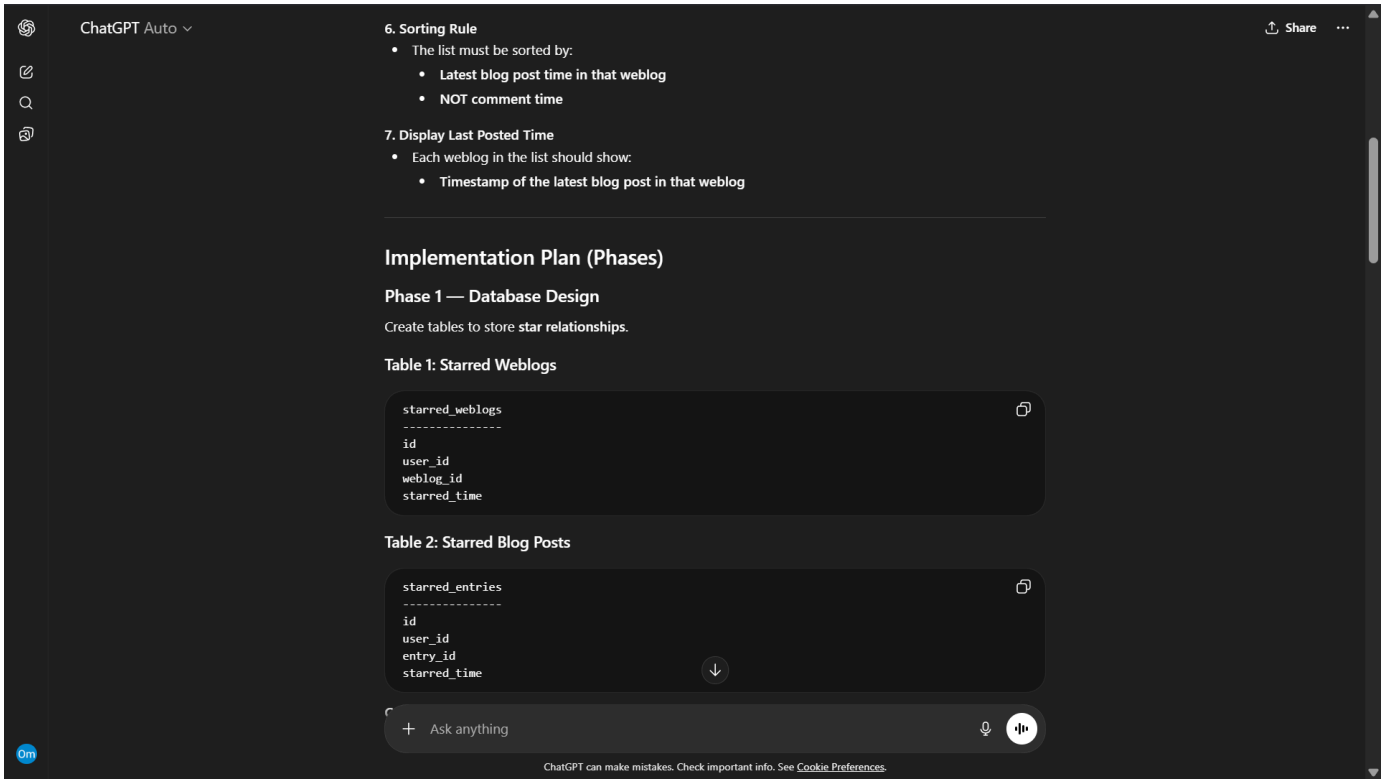
How this helped in code:

- Broke Task 1A into concrete actions (star/unstar weblog, star/unstar entry, homepage listing, sorting rule).
- Confirmed that sorting must use latest blogpost time and explicitly ignore comment timestamps.

Evaluation:

- Correct high-level interpretation of assignment requirements.
- Needed project-specific implementation mapping in Roller.

2) Phase 1 database design suggestion



How this helped in code:

- Proposed table-level constraints for duplicate prevention and star relationship storage.
- Helped guide index and uniqueness thinking.

Evaluation:

- Suggested two-table approach (`starred_weblogs`, `starred_entries`) was useful conceptually.
- Final implementation was manually adapted to a unified model (`roller_star`) for better integration.

3) Service and DAO layer guidance

ChatGPT Auto

Share

Constraints

- `(user_id, weblog_id)` unique
- `(user_id, entry_id)` unique

Purpose:

- Prevent duplicate stars.

Phase 2 — Backend Service Layer

Add services in Roller backend.

Example service methods:

```
starWeblog(userId, weblogId)
unstarWeblog(userId, weblogId)

starEntry(userId, entryId)
unstarEntry(userId, entryId)

getStarredWeblogs(userId)
getStarredEntries(userId)
```

Responsibilities:

- Validate user
- Insert/delete star records
- Fetch starred items

Phase 3 — DAO Layer

Create DAO methods.

Example:

Ask anything

ChatGPT can make mistakes. Check important info. See Cookie Preferences.

How this helped in code:

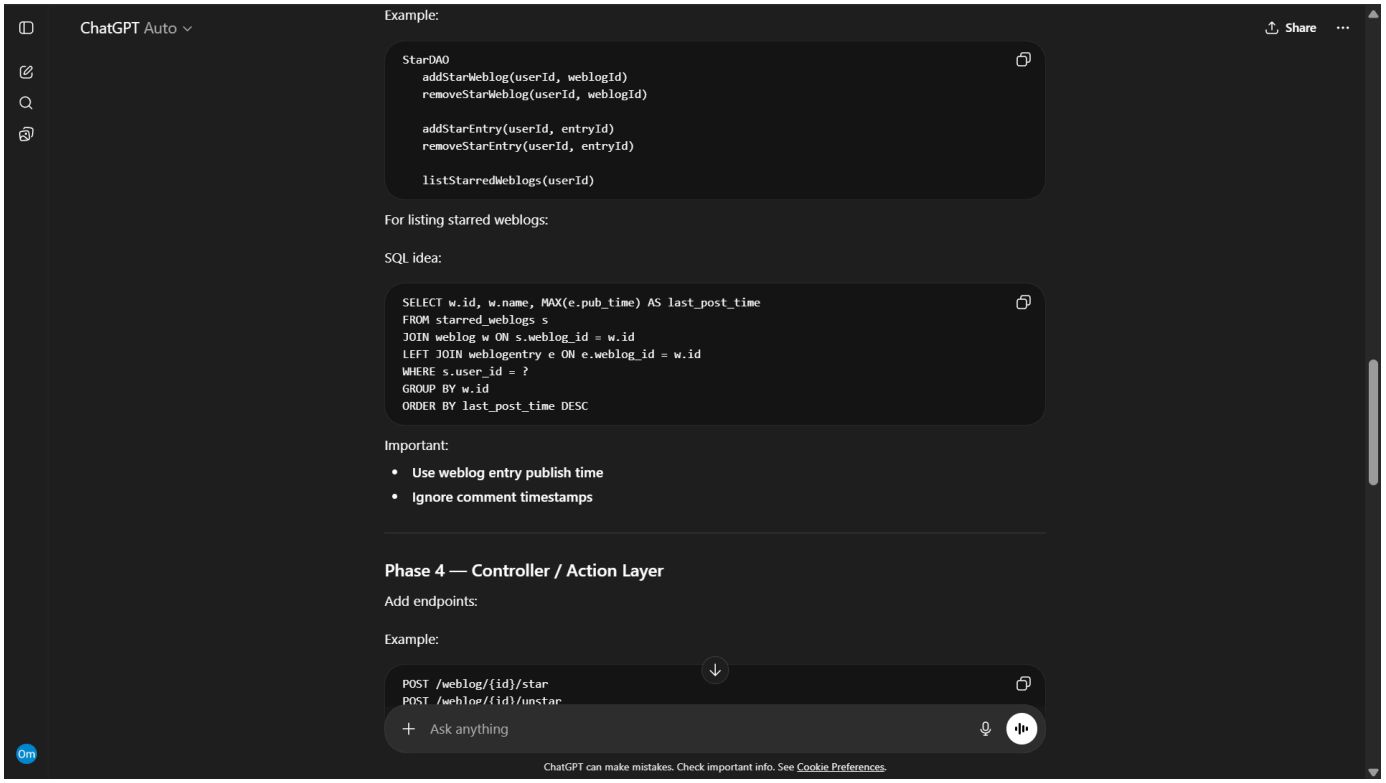
- Suggested service-level method set for star/unstar/list operations.
- Reinforced clean separation between service logic and data-access logic.

Evaluation:

- Useful for modular layering decisions.
- DAO/service signatures were manually revised to match existing Roller architecture.

---

4) Query logic and action endpoint suggestions



How this helped in code:

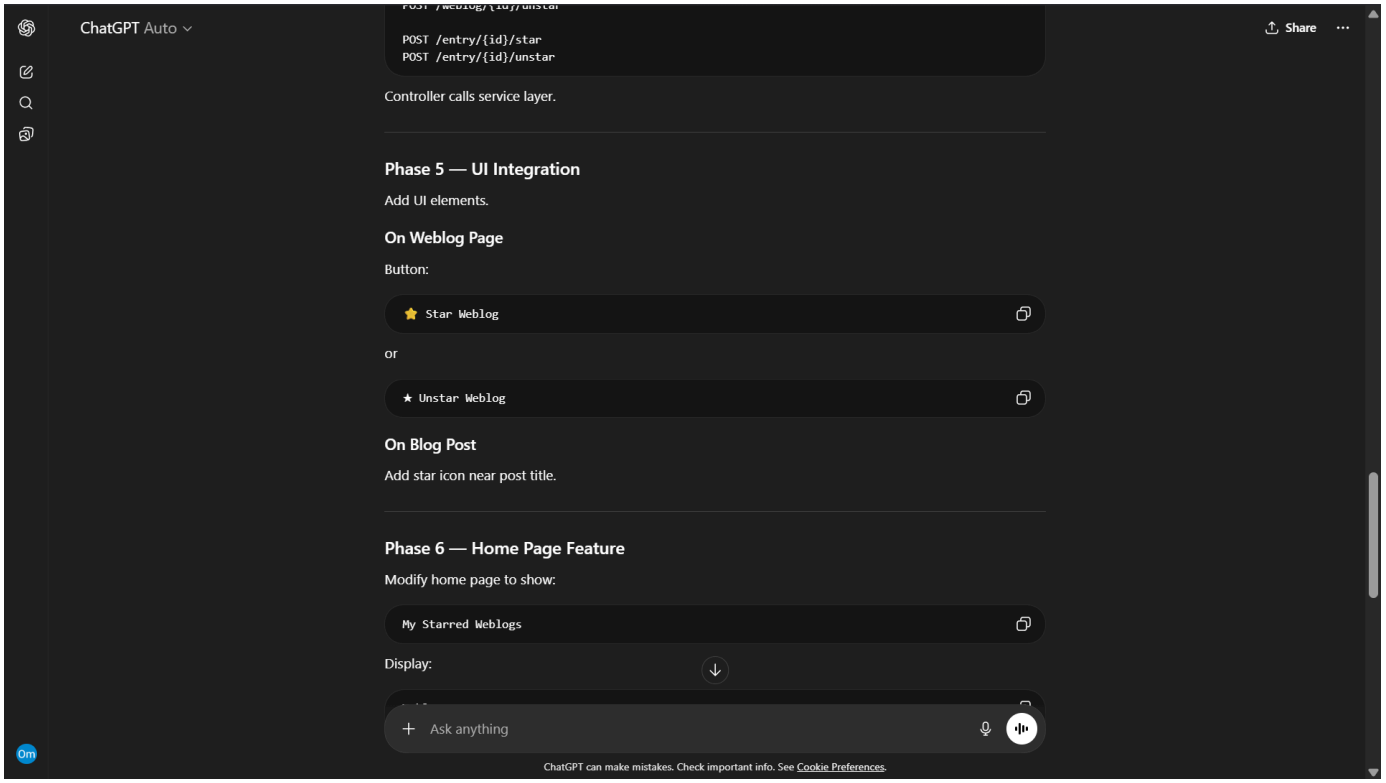
- Provided sample SQL-style aggregation for latest-post-time ordering.
- Suggested controller/action endpoint flow for star/unstar operations.

Evaluation:

- Correct idea for `MAX(post_time)` ordering.
- Query syntax and integration details required manual correction in JPA/service implementation.

---

5) UI integration and homepage section suggestions



How this helped in code:

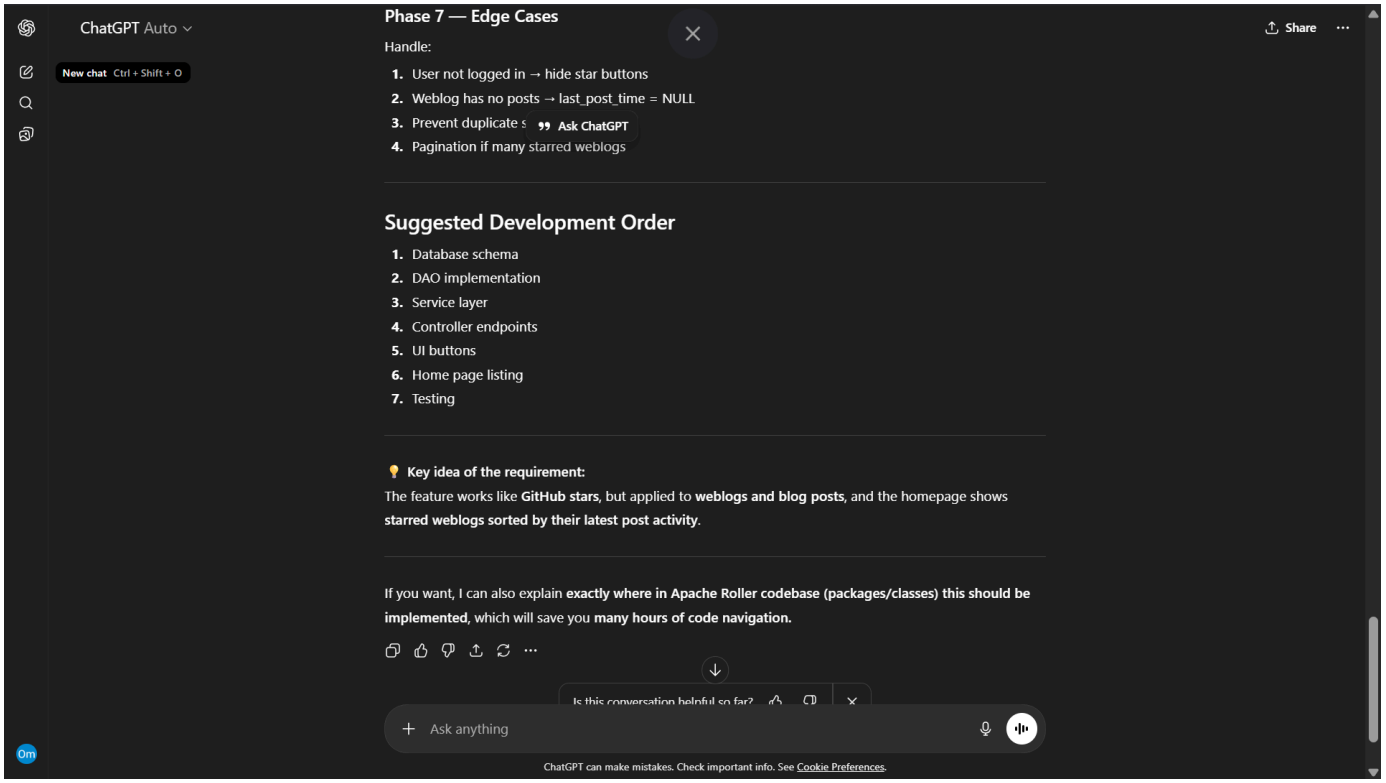
- Helped decide where to place Star/Unstar controls in theme templates.
- Helped define homepage section behavior for showing starred weblogs.

Evaluation:

- Useful for UI flow and empty-state handling.
- Final rendering details were manually adapted to Roller Velocity/JSP theme structure.

---

6) Edge cases and development order checklist



How this helped in code:

- Highlighted practical concerns (duplicate stars, no-post weblogs, visibility conditions).
- Suggested implementation sequence used as a validation checklist.

Evaluation:

- Good checklist for final verification.
- Final behavior and compatibility were confirmed through manual testing and project-level integration checks.

Final Accuracy Summary

- LLM was useful for **decomposing requirements, proposing persistence/service structure, and suggesting UI integration flow**.
- LLM output was **not copied as-is**; it required manual adaptation for Roller-specific classes, mappings, and action wiring.
- Final implementation decisions were manually validated against Task 1A requirements:
  - Star/unstar support for weblogs and blogposts
  - Home-page listing of starred weblogs
  - Sorting by most recent blogpost update time
  - Display of last posted date/time
  - Compatibility with existing UI/action/service architecture

Task 1B

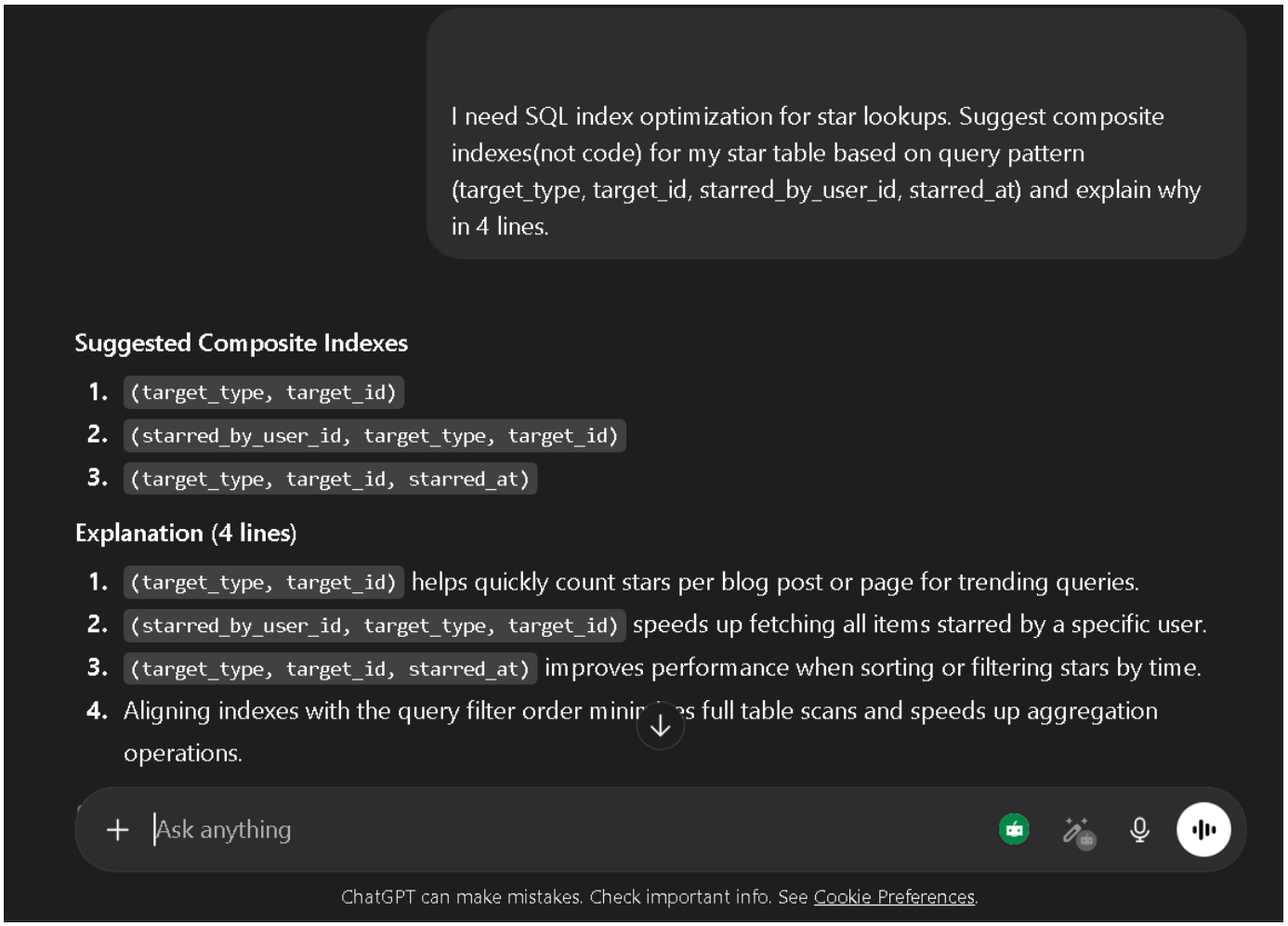
Purpose

This document records how LLM assistance was used while implementing **Task 1B (Trending Blogs and Blog Posts)**.

The screenshots below capture the prompt/response workflow used during coding, and the evaluation notes summarize what was correct, what required fixes, and what was finally adopted.

Screenshot Evidence

1) Initial implementation guidance



How this helped in code:

- Helped structure the first version of trending retrieval logic.
- Helped decide that ranking should be based on user stars.

Evaluation:

- Correct direction on using star-based ranking.
- Needed manual adjustment to match project constraints exactly (Top 5 and efficient query-based retrieval).

---

2) Query optimization and sorting refinement

Add fallback-safe UI logic in template: if trending list is empty, show "No trending items yet". Explain me how to write Velocity code snippet.

Apache Roller uses **Apache Velocity templates (.vm files)**.
You need a **simple conditional check** to see whether the trending list is empty. If empty → show fallback message.

Basic Idea

Velocity provides:

```
#if
#else
#foreach
```

You check:

```
if list exists AND size > 0
```

Ask anything

ChatGPT can make mistakes. Check important info. See Cookie Preferences.

How this helped in code:

- Helped refine DB-side aggregation logic.
- Helped enforce deterministic ordering when counts are equal.

Evaluation:

- Correctly emphasized aggregation and ordering.
- Some query snippets required syntax and type fixes while integrating into JPA service methods.

---

3) UI-level integration suggestions

You check:

```
if list exists AND size > 0
```

Otherwise show fallback text.

Velocity Logic Structure

The structure looks like:

```velocity
#if(trendingList && $trendingList.size() > 0)
   show trending items
#else
   show fallback message
#end
```

Explanation:

- `trendingList` → list passed from controller/service
- `$trendingList.size()` → number of items
- `#foreach` → iterate items

Ask anything

ChatGPT can make mistakes. Check important info. See Cookie Preferences.

How this helped in code:

- Helped with display strategy for trending sections in templates.
- Helped with safe rendering behavior for missing/empty data.

Evaluation:

- Useful for UI wiring and fallback behavior.
- Final Velocity integration was manually adapted to existing Roller theme structure.

---

4) Debug and edge-case guidance

I am getting wrong ranking in trending results: some weblogs with same star count are appearing in random order. My query currently orders only by COUNT. Fix query so it orders by COUNT(DISTINCT user) DESC and then MAX(starredAt) DESC deterministically.

You are seeing random ordering because **multiple weblogs have the same star count**, and SQL does not guarantee order when the `ORDER BY` values are equal. You need a **secondary sorting condition**.

Fixed Query Logic

Order first by:

1. **Number of unique users who starred** → `COUNT(DISTINCT starred_by_user_id) DESC`
2. **Most recent star time** → `MAX(starred_at) DESC`

Correct Query

SQL

```sql
SELECT target_id,
       COUNT(DISTINCT starred_by_user_id) AS star_count,
       MAX(starred_at) AS last_star_time
FROM stars
WHERE target_type = 'WEBLOG'
GROUP BY target_id
ORDER BY star_count DESC, last_star_time DESC
LIMIT 5;
```

How this helped in code:

- Helped identify practical edge cases (ties, empty results, star visibility).
- Helped shape final verification checklist.

Evaluation:

- Good debugging checklist.
- Final correctness validated through manual feature testing in multiple themes.

Final Accuracy Summary

- LLM was useful for **query structure, ordering strategy, and template-level wiring ideas**.
- LLM output was **not copied as-is**, it required manual correction for project-specific classes and compatibility.
- Final implementation decisions were manually validated against assignment requirements:
  - Top 5 trending weblogs and entries
  - Distinct-user star counting
  - Efficient DB-side aggregation
  - Correct UI rendering in integrated themes

Task 2

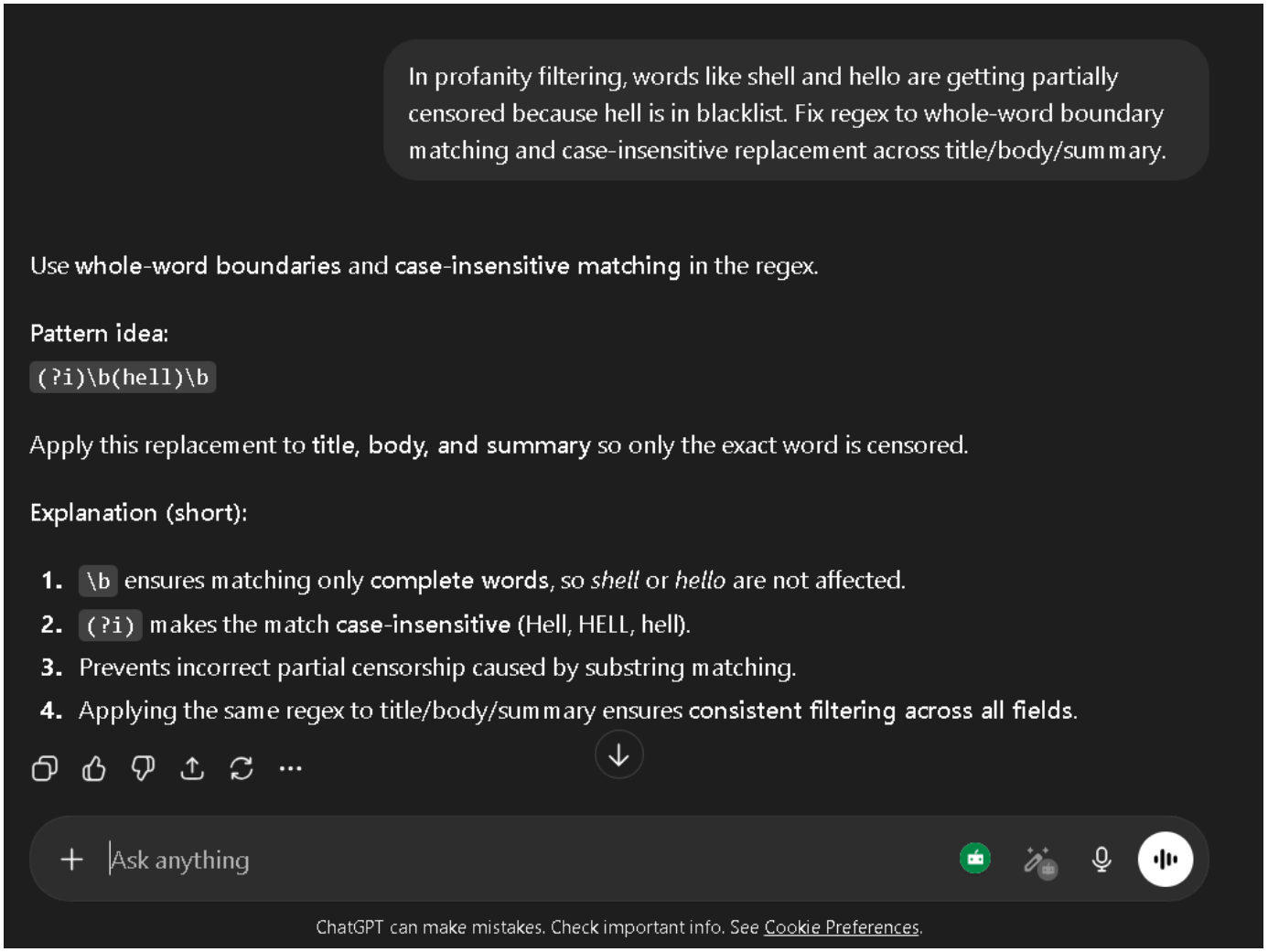
Purpose

This document records how LLM assistance was used while implementing **Task 2 (Transforming Feeds)**.

The screenshots below capture the prompt/response workflow used during coding, and the evaluation notes summarize where LLM output was useful and where manual corrections were applied.

Screenshot Evidence

1) Pipeline architecture planning



How this helped in code:

- Helped define a clean processing flow before save.
- Helped split processing into independent steps.

Evaluation:

- Correct high-level architecture guidance.
- Required adaptation to Roller action/save lifecycle.

---

2) Step-level implementation ideas

Auto-tag generator is adding duplicates and noisy terms like this, that, with. Improve algorithm with stopword removal, min token length, lowercase normalization, and duplicate-safe merge with existing tags.

Improve the tagging pipeline with four preprocessing steps before generating tags.

Improved approach:

1. **Lowercase normalization**
   Convert all tokens to lowercase so `AI`, `ai`, and `Ai` become the same tag.
2. **Stopword removal**
   Remove common words like *this, that, with, the, is* using a predefined stopword list.
3. **Minimum token length filter**
   Ignore tokens shorter than a threshold (e.g., `< 3 characters`) to avoid noisy terms.
4. **Duplicate-safe merge with existing tags**
   Store tags in a **set-like structure** before saving so repeated tags are automatically removed and existing tags are preserved.

Why this works (short):

- Stopword filtering removes meaningless words.
- Length filtering eliminates very short noisy tokens.
- Lowercasing ensures consistent tag representation.
- Set-based merging prevents duplicates and keeps tags clean.

Ask anything

ChatGPT can make mistakes. Check important info. See Cookie Preferences.

How this helped in code:

- Helped draft behavior for profanity filtering and length limiting.
- Helped shape configuration-driven behavior.

Evaluation:

- Useful as a baseline.
- Regex details and field handling (title/body/summary) required manual fixes.

---

3) Extensibility and toggles

Show how to integrate pipeline into entry save flow in Struts action: call pipeline before saveWeblogEntry(entry), log step failures, and avoid breaking save on non-critical step errors.

You integrate the **processing pipeline inside the Struts Action before saving the entry**. The idea is: run the pipeline → catch/log step errors → still continue with save if the step is non-critical.

Flow in Struts Action

Java

```java
// Inside WeblogEntrySaveAction or similar Struts action

public ActionForward saveEntry(ActionMapping mapping, ActionForm form,
                               HttpServletRequest request,
                               HttpServletResponse response) throws Exception {

    WeblogEntry entry = buildEntryFromForm(form);

    try {
        if(feedPipeline != null) {
            feedPipeline.process(entry);   // run preprocessing steps
        }
    } catch (Exception e) {
        log.warn("Feed pipeline step failed but continuing save: " + e.getMessage(), e);
        // do NOT stop save since steps are non-critical
    }

    // proceed with normal Roller save
    weblogEntryManager.saveWeblogEntry(entry);

    return mapping.findForward("success");
}
```

Ask anything

ChatGPT can make mistakes. Check important info. See Cookie Preferences.

How this helped in code:

- Helped define per-step enable/disable keys.
- Reinforced add/remove-step design with minimal impact on other steps.

Evaluation:

- Correct modularity direction.
- Final property names and defaults were adjusted to project conventions.

---

4) Debugging and robustness checks

Java

Pipeline Step Failure Handling

Inside each pipeline processor:

Java

```java
public void process(WeblogEntry entry) {
    try {
        handle(entry);      // profanity filter / tagging / summary
    } catch (Exception e) {
        log.warn("Processor failed: " + this.getClass().getSimpleName(), e);
    }

    if(next != null) {
        next.process(entry);
    }
}
```

Execution Flow

```
User submits blog
        ↓
Struts Action (SaveEntryAction)
        ↓
FeedProcessingPipeline.process(entry)
        ↓
[ProfanityFilter → Summary → TagGenerator]
        ↓
saveWeblogEntry(entry)
```

Ask anything

ChatGPT can make mistakes. Check important info. See Cookie Preferences.

How this helped in code:

- Helped identify failure-handling expectations (non-critical step issues should not crash save).
- Helped list integration tests for end-to-end pipeline behavior.

Evaluation:

- Good test and resilience suggestions.
- Final save flow behavior was manually verified using real entry posting in Roller UI.

Final Accuracy Summary

- LLM was useful for **pipeline decomposition, step responsibilities, and configuration strategy**.
- LLM output was **partially correct** and required manual code-level corrections.
- Final implementation was validated against assignment goals:
  - Multi-step processing pipeline
  - Config-based enable/disable support
  - Easy step add/remove behavior without breaking other steps

Task 3 - Bug Report

Screenshot Evidence

1) Clarify Task 3 requirements into a checklist

Prompt:

> "I already made my own Task 3 plan. Can you quickly list the must-have requirements so I can cross-check I didn't miss anything, especially email triggers?"

Screenshot:

I already made my own Task 3 plan. Can you quickly list the must-have requirements so I can cross-check I didn't miss anything, especially email triggers?

Roller Project 2 – Task 3: Bug Reports

Must-Have Requirements (Quick Cross-Check)

Backend (Core)

- BugReport entity with fields: `id`, `title`, `description`, `status`, `priority`, `reported_by`, `assigned_to`, `created_at`, `updated_at`.
- APIs for:
  - Create bug
  - View bug list and bug details
  - Update bug (status, priority, assignment)
- Status workflow support (e.g., `OPEN → IN_PROGRESS → RESOLVED → CLOSED`).
- Input validation for required fields and valid status/priority values.
- Store timestamps for creation and updates.

UI (Essential Screens)

- **Bug List Page:** Table showing title, status, priority, reporter, assignee.
- **Create Bug Form:** Title, description, priority, optional assignee.
- **Bug Details Page:** Full bug info with ability to update status or assignment.
- **Basic filtering/search** by status or priority.

Email Triggers (Important Cross-Check)

Event	Email Recipients
Bug Created	Project admin/manager, assigned developer (if assigned)
Bug Assigned	Assigned developer
Status Updated	Bug reporter + assigned developer
Bug Resolved	Bug reporter

Roles (Basic)

- **Reporter:** create and view bugs.
- **Developer:** update status of assigned bugs.
- **Admin/Manager:** assign bugs and manage priorities.

✅ If these are covered, you should have **all the essential requirements for Task 3**.

What was correct / useful: The response correctly summarized the key implementation requirements for the bug reporting feature, including the core backend elements (BugReport entity, APIs for creating, viewing, and updating bugs), essential UI components (bug list, bug creation form, and bug details page), and the main workflow for bug status updates.

What was incomplete / adjusted by us: The response provided a generalized checklist and did not include project-specific implementation details from the Apache Roller codebase, such as exact classes, services, or integration points where the bug report feature should be implemented. Some elements like role permissions, exact UI layout, and how email services integrate with the existing Roller infrastructure needed to be clarified and aligned with the project's architecture.

Net evaluation: The response was useful as a quick high-level checklist to validate that the major requirements were not missed, particularly the email trigger conditions and core feature components. However, it required refinement and adaptation to fit the specific structure and constraints of the Apache Roller codebase. Overall, the response served as a helpful starting point but needed manual adjustments for accurate implementation.

---

2) How to keep notifications modular for future Slack/Teams

Why I asked: I already had an event + channel idea in mind, and I just wanted a quick sanity check before coding.

Prompt :

> "I'm thinking event + dispatcher + channel interface for notifications. Email first, maybe Slack/Teams later. Does this sound reasonable or am I missing something basic?"

Screenshot:

I'm thinking event + dispatcher + channel interface for notifications. Email first, maybe Slack/Teams later. Does this sound reasonable or am I missing something basic?

Yes, that design is **reasonable and actually a good extensible approach**. For a notification system (email now, Slack/Teams later), **event + dispatcher + channel interface** is a common pattern. You're not missing anything fundamental, but there are a few **basic things you should ensure are included**.

Below is a **quick cross-check checklist**.

---

1. Core Architecture (Your Idea)

Your structure should look roughly like this:

Event → Dispatcher → Notification Channel

Example flow:

```
BugCreatedEvent
      ↓
NotificationDispatcher
      ↓
EmailChannel (now)
SlackChannel (future)
TeamsChannel (future)
```

This is good because:

- **Loose coupling**
- Easy to add new notification channels
- Business logic separated from delivery mechanism

---

2. Must-Have Components

1️⃣ Event Classes

Events should represent **system actions**.

Examples:

```
BugCreatedEvent
BugAssignedEvent
BugStatusChangedEvent
BugCommentAddedEvent
```

Each event should contain:

- bugId
- reporter
- assignee
- event metadata

---

2. Bug Assigned

Trigger: Developer assigned to bug

Email sent to:

- Assigned developer

---

3. Status Changed

Trigger: Bug status updated (e.g., RESOLVED, CLOSED)

Email sent to:

- Bug reporter
- Assigned developer

---

4. Comment Added

Trigger: New comment on bug

Email sent to:

- Bug reporter
- Assigned developer
- Previous commenters (optional but common)

UI Requirements

- Bug submission form
- Bug list page
- Bug detail page
- Comment section
- Status update controls

Backend Requirements

- Bug model / database schema
- Comment model
- Email service integration
- API endpoints for:
  - create bug
  - update bug
  - assign bug
  - add comment
  - fetch bugs

✅ **Minimum email triggers you should definitely include:**

1. Bug created
2. Bug assigned
3. Status changed
4. Comment added

Ask anything

What was correct / useful: The response suggested using an event-based notification system with a dispatcher and a channel interface. This was useful because it keeps the notification logic decoupled from the main application logic and makes it easier to extend the system later. The idea of having separate channels (like Email, Slack, or Teams) implementing a common interface also aligned well with our intended design.

What was incomplete / adjusted by us: Some practical aspects needed refinement when translating the suggestion into our plan. For example, the response mentioned concepts like async queues, template services, and advanced configuration, which were not necessary for our assignment scope.

Net evaluation: Overall, the response was helpful in confirming that our planned architecture was reasonable and in highlighting a clean event-driven structure for notifications. However, it required some simplification and adjustment to match the assignment requirements and keep the implementation manageable.

---

3) How to send emails safely without breaking the DB transaction

Why I asked: I was about to wire the email calls and wanted one quick confirmation on failure handling.

Prompt:

> "Quick check: if mail sending fails, I still want bug save/update to succeed. What's the safest way to handle this?"

Screenshots:

Safest approach is **don't let notification failures affect the main transaction** (bug creation/update). Treat email as a **secondary side-effect**, not part of the core operation.

Here's the practical pattern.

---

1. Save Bug First (Main Transaction)

The **bug operation must complete successfully regardless of email status**.

Flow:

```
Create / Update Bug
      ↓
Commit to database
      ↓
Trigger notification
```

So even if email fails, the **bug is already saved**.

Pseudo idea:

```
saveBug(bugData)

try:
    dispatcher.dispatch(BugCreatedEvent)
catch:
    log error
```

Key point:
Never roll back the bug save because email failed.

---

2. Wrap Notification in Try–Catch

The simplest safe method.

```
try {
    notificationDispatcher.dispatch(event)
} catch (Exception e) {
    log.error("Notification failed", e)
}
```

Result:

Situation	Outcome
Bug save success + email success	ideal
Bug save success + email fails	bug still saved
Bug save fails	whole request fails

This is usually **enough for coursework systems**.

---

3. Log the Failure

Always log it so the issue can be debugged later.

Example:

```
Notification failed for Bug #1023
```

```
Notification failed for Bug #1023
Reason: SMTP timeout
```

Logging prevents **silent failures**.

---

4. (Better Design) Async Notifications

A safer real-world approach:

```
Bug Saved
   ↓
Publish Event
   ↓
Queue
   ↓
Worker sends email
```

Benefits:

- Request stays fast
- Email failures don't affect user request
- Retry possible

But this is **optional for your assignment**.

---

5. Optional Retry (Nice but not required)

If email fails:

```
retry 3 times
```

or mark notification as failed.

Again, not required unless project demands reliability.

Recommended Minimal Design for Your Task

```
saveBug()

try:
   dispatchNotification()
catch:
   logFailure()
```

Rules:

- Bug operation **must succeed independently**
- Notification **must not break the request**
- Failure should be **logged**

✅ **One-line answer suitable for report:**

> Email sending is treated as a non-critical side effect. Bug creation or updates are committed to the database first, and notification dispatch is executed in a try-catch block so that any email failure is logged but does not affect the main operation.

If you want, I can also give you a **2–3 line casual version for the LLM usage report**, which usually looks more natural than the formal one.

What was correct / useful: The response correctly suggested separating the email notification logic from the main bug save/update operation. It recommended handling notifications as a secondary action and wrapping the email sending in a try–catch block so that even if the email fails, the main operation (saving or updating the bug) still succeeds.

What was incomplete / adjusted by us: The response also mentioned more advanced options like using asynchronous queues and retry mechanisms for sending emails. While these are good practices in real-world systems, they were not necessary for the scope of our assignment.

Net evaluation: Overall, the response was helpful in highlighting the correct way to isolate notification failures from the main workflow. It gave a clear direction for handling errors safely, though we simplified some of the suggested ideas to keep the implementation straightforward for the project.

---

4) Struts2 action structure (submit/list/admin dashboard)

Why I asked: I had already decided the pages, but I wanted a quick check that the split looked clean and not over-engineered.

Prompt :

> "I'm planning 3 flows only: submit bug, my-bugs list (+ delete), and admin dashboard (+ status update/delete). Is this a good minimal split?"

Screenshots:

I'm planning 3 flows only: submit bug, my-bugs list (+ delete), and admin dashboard (+ status update/delete). Is this a good minimal split?

Yes, that's a **good minimal split** for Task 3. It keeps the system simple while still covering the core bug-tracking workflow. For an assignment, these **three flows are usually sufficient**.

Here's a quick sanity check.

---

1. Submit Bug

User reports a new bug.

Typical actions:

- Fill bug form (title, description, steps, severity, etc.)
- Save bug in DB
- Status set to **OPEN**
- Bug linked to **reporter**

Optional but good:

- Trigger **BugCreated event → email notification**

This flow basically covers **bug creation**.

---

2. My Bugs (User View)

User manages bugs they reported.

Features:

- List bugs reported by the user
- Show:
  - bug title
  - status
  - priority
  - created date
- Allow:
  - **delete bug** (if allowed)
  - view details

This flow covers **bug tracking from reporter side**.

---

3. Admin Dashboard

Admin or maintainer manages bugs.

Features:

- View **all bugs**
- Update:
  - status (OPEN → IN_PROGRESS → RESOLVED → CLOSED)
- Delete bug if necessary

Optional but useful:

- Status change → **send notification**

This flow covers **bug management / triage**.

Ask anything

3. Admin Dashboard

Admin or maintainer manages bugs.

Features:

- View **all bugs**
- Update:
  - status (OPEN → IN_PROGRESS → RESOLVED → CLOSED)
- Delete bug if necessary

Optional but useful:

- Status change → **send notification**

This flow covers **bug management / triage**.

Minimal Architecture Coverage

With these 3 flows you still cover the main system concepts:

Feature	Covered?
Bug creation	✓
Bug listing	✓
Bug status management	✓
Role separation (user/admin)	✓
Notifications	✓
CRUD operations	✓

So from a **design perspective**, it's perfectly valid.

Small Thing You Might Add (Optional)

A simple **bug detail page**:

```
My Bugs
   ↓
Click bug
   ↓
Bug details page
```

But this is **nice-to-have**, not required.

✅ **Final verdict:**
Your split

```
1. Submit Bug
2. My Bugs (list + delete)
3. Admin Dashboard (status update + delete)
```

is a **clean minimal design** and totally reasonable for the assignment.

If you want, I can also show you a **very small system design diagram (5 components)** that makes this look **much more professional in reports/interviews**

Ask anything

ChatGPT can make mistakes. Check important info. See Cookie Preferences

What was correct / useful: The response confirmed that splitting the functionality into three main flows — submitting a bug, viewing a “my bugs” list, and having an admin dashboard — is a reasonable minimal design. It helped validate that these flows are enough to cover the basic lifecycle of bug reporting and management.

What was incomplete / adjusted by us: While the response suggested a few additional ideas like having a separate bug detail page or extra optional features, these were not strictly necessary for the scope of our task.

Net evaluation: Overall, the response was helpful mainly as a quick validation that our planned structure was reasonable and not missing any essential components. Some of the additional suggestions were useful to think about, but we simplified the design to keep it aligned with the assignment requirements.

Task 4 - Admin Dashboard

Purpose

This document records how LLM assistance was used while implementing **Task 4**.

The screenshots below capture the prompt/response workflow used during coding, and the evaluation notes summarize where LLM output was useful and where manual corrections were applied.

Screenshot Evidence

1) High-level design and planning

give in proper short

Add a **Feature Toggle mechanism** to control whether the Admin Dashboard (Task 4) is enabled.

Architecture:
Introduce a `FeatureToggleService` that checks configuration before calling `DashboardService`.

Configuration sources:

- **Site-level config:** stored in site settings (e.g., `admin_dashboard_enabled`).
- **User-level preference:** stored in user preferences (e.g., `dashboard_enabled`).
  User preference overrides site setting.

Flow:
Controller → `FeatureToggleService.isDashboardEnabled(user)` → if true build dashboard, else hide feature.

Benefits:
Allows enabling/disabling the dashboard per site or user without code changes, improving flexibility and safe rollout.

Ask anything

ChatGPT can make mistakes. Check important info. See Cookie Preferences.

How this helped in code:

- Helped outline the feature architecture and integration points.
- Helped identify which existing services and templates to extend.

Evaluation:

- Correct high-level guidance.
- Required adaptation to project-specific lifecycle and naming conventions.

---

2) Step-level implementation suggestions

List common edge cases and failure modes for [Task 4 feature]. How should errors be handled gracefully?

Common Edge Cases / Failure Modes (Admin Dashboard – Task 4)

1. **No data available**
   Example: no users, posts, or stars yet.
   Handling: return default values (0, "No data available") and display a fallback message in UI.
2. **Database query failure or timeout**
   Metrics queries may fail due to DB issues.
   Handling: catch exceptions in `DashboardService`, log the error, and show partial dashboard with available metrics.
3. **Metric computation failure**
   One metric provider may fail while others work.
   Handling: isolate metrics (Strategy pattern) and skip the failed metric instead of breaking the whole dashboard.
4. **Unauthorized access**
   Non-admin users trying to access the dashboard.
   Handling: perform role checks in the controller and redirect to an error or access-denied page.
5. **Feature disabled by configuration**
   Dashboard feature may be turned off via feature toggle.
   Handling: check configuration first and show "Dashboard not available" message.

Graceful Error Handling Approach

- Use **try–catch blocks in the service layer**.
- **Log failures** for debugging (`warn` or `error`).
- **Return partial results or fallback values** instead of crashing the page.
- Ensure the **UI handles null or empty metrics safely**.

Ask anything

ChatGPT can make mistakes. Check important info. See Cookie Preferences.

How this helped in code:

- Proposed candidate algorithms and helper methods.
- Suggested configuration keys and default values.

Evaluation:

- Useful baseline code and configuration ideas.
- Regex/edge-case details required manual refinement.

---

3) Testing and validation

Show how to extend the processing pipeline to include [Task 4 feature] as a modular step, following the project's conventions.

To add **Task 4 (Admin Dashboard metrics collection)** as a modular step, extend the existing **processing pipeline** so a dashboard metrics step can run **after a weblog entry is saved or updated**. This keeps the feature modular and consistent with the project's **pipeline/handler convention (Chain of Responsibility)**.

---

1. Create a Pipeline Step Interface

All pipeline steps follow a common interface.

Java

```java
public interface EntryProcessingStep {
    void process(WeblogEntry entry) throws ProcessingException;
}
```

This keeps each step independent and interchangeable.

---

2. Implement Dashboard Metrics Step

Create a step that updates dashboard metrics (e.g., activity counters, category stats).

Java

```java
public class DashboardMetricsStep implements EntryProcessingStep {

    private DashboardService dashboardService;

    public DashboardMetricsStep(DashboardSer e dashboardService) {
        this.dashboardService = dashboardService;
    }
```

Ask anything

ChatGPT can make mistakes. Check important info. See Cookie Preferences.

How this helped in code:

- Recommended unit and integration test cases.
- Suggested how to mock external dependencies for tests.

Evaluation:

- Test outlines were broadly correct.
- Test data and assertions were adjusted to match real responses.

---

4) Debugging and hardening

2. Implement Dashboard Metrics Step

Create a step that updates dashboard metrics (e.g., activity counters, category stats).

Java

```java
public class DashboardMetricsStep implements EntryProcessingStep {

    private DashboardService dashboardService;

    public DashboardMetricsStep(DashboardService dashboardService) {
        this.dashboardService = dashboardService;
    }

    @Override
    public void process(WeblogEntry entry) {
        try {
            dashboardService.updateMetrics(entry);
        } catch (Exception e) {
            log.warn("Dashboard metrics step failed for entry " + entry.getId(), e);
        }
    }
}
```

Responsibilities:

- update statistics used by the admin dashboard
- log failures without stopping the pipeline

---

3. Register Step in Pipeline

Extend the existing pipeline initialization.

Example:

Ask anything

ChatGPT can make mistakes. Check important info. See Cookie Preferences.

How this helped in code:

- Suggested failure modes and graceful degradation strategies.
- Helped define logging and monitoring points.

Evaluation:

- Good resilience recommendations.
- Final logging levels and messages were tuned manually.

Final Accuracy Summary

- LLM was useful for **architecture decomposition, configuration strategy, and test scaffolding**.
- LLM output was **partially correct** and required manual code-level corrections and integration adjustments.
- Final implementation validation checklist:
  - Feature integrated into existing pipeline
  - Config-based toggles supported
  - Unit and integration tests added

Task - 5

Purpose

This document records how LLM assistance was used while implementing **Task 5 (Web Translation & Caching)**.

---

1) Root-cause diagnosis for unresolved `${env...}` placeholders

Prompt used:

```
I have a Java webapp. When the translation feature runs it logs:
"Illegal character in path at index 1: ${env.TRANSLATION_SARVAM_API_URL}".
Relevant file:
app/src/main/java/org/apache/roller/weblogger/business/translation/Transla
tionService.java (provider factory uses placeholders).
I attempted to inspect environment variables and .env. Please explain the
most likely root cause and list 2–3 minimal diagnostic steps I can run
locally (commands or small checks). Do NOT modify my code automatically —
only explain what to check and why.
```

I have a Java webapp. When the translation feature runs it logs:
"Illegal character in path at index 1: ${env.TRANSLATION_SARVAM_API_URL}".
Relevant file: app/src/main/java/org/apache/roller/weblogger/business/translation/TranslationService.java (provider factory uses placeholders).
I attempted to inspect environment variables and .env. Please explain the most likely root cause and list 2–3 minimal diagnostic steps I can run locally (commands or small checks). Do NOT modify my code automatically — only explain what to check and why.

TranslationService.java:1-676

✓ Reviewed diagnostic steps and captured server logs

I'll add diagnostic todo items to the task list, then summarize likely root cause and give 3 quick local checks.

✓ Created 6 todos

Progress: added diagnostic todos (IDs 4–6). Next: root cause and 3 minimal checks.

Likely root cause

- The app is receiving the literal placeholder string "${env.TRANSLATION_SARVAM_API_URL}" as the Sarvam API URL because the runtime environment variable was not set (or .env fallback wasn't loaded into the JVM). The HTTP client then tries to parse that literal as a URL and throws "Illegal character in path".

Minimal diagnostic steps (run locally)

1. Check environment variables in the shell and in the running JVM process

- In your shell (quick check):

```
env | grep -E 'SARVAM|TRANSLATION_SARVAM'
```

- If Jetty/Tomcat was started outside your shell (systemd, IDE, separate terminal), find the JVM PID and inspect its environment:

```
# find PID (adjust pattern if needed)
pgrep -f 'jetty|org.eclipse.jetty|java' | head -n 5

# then (replace <pid> with the PID)
tr '\0' '\n' < /proc/<pid>/environ | grep -E 'SARVAM|TRANSLATION_SARVAM'
```

Why: confirms whether the process actually sees TRANSLATION_SARVAM_API_URL or SARVAM_API_KEY.

2. Search the deployed/built config for the literal placeholder

- Look for the placeholder string in classpath/config files (shows whether a properties file still contains the ${env...} text):

```
# run from project root
grep -R --line-number '\${env\.TRANSLATION_SARVAM_API_URL}' .
```

- Also check the resolved properties in the exploded target/classes (if running from mvn/jetty):

How this helped in code:

- Confirmed likely cause: unresolved placeholder passed into provider URL construction.
- Focused checks on env resolution path before changing Java logic.
- Avoided premature code edits and reduced debug scope.

Evaluation:

- High value for narrowing issue quickly.
- Good for triage and minimal reproducible diagnostics.

---

2) Manual smoke-test checklist for .env fallback + runtime switching

Prompt used:

```
Give a short manual smoke-test checklist (3–6 steps) to validate: .env
fallback works, Sarvam requests succeed, provider dropdown toggles
providers without JVM restart, and cache keys include provider. Include
the exact curl or mvn commands to run and the response JSON fields to
inspect.
```

Give a short manual smoke-test checklist (3–6 steps) to validate: .env fallback works, Sarvam requests succeed, provider dropdown toggles providers without JVM restart, and cache keys include provider. Include the exact `curl` or `mvn` commands to run and the response JSON fields to inspect.

TranslationService.java:1-676

> Used 2 references

✓ Analyzed cache size options and suggested log monitoring

Steps to smoke-test (.env fallback, Sarvam success, runtime provider switch, cache keys include provider)

1. Start the app (if not running)

```
cd /home/gaurav/se_main/project-1-team-27
mvn -pl app -DskipTests jetty:run
```

2. Check metadata (.env resolution)

```
curl -s http://localhost:8080/roller-services/translate | jq .
```

Inspect JSON fields:

- provider → should be "sarvam" (not a literal "${env...}")
- providers → should include ["sarvam","gemini"]
  If provider equals a literal ${env...} then .env/env-vars are unresolved.

3. Test Sarvam translate request

```
curl -s -X POST http://localhost:8080/roller-services/translate \
  -H 'Content-Type: application/json' \
  -d '{"provider":"sarvam","source":"auto","target":"te","texts":["Hello world"]}' | jq .
```

Inspect:

- translations (array) exists
- translations.length == texts.length
- translations[0] should NOT equal "Hello world" (if equal, Sarvam returned original / failed)

4. Test runtime provider switch (no JVM restart)

```
# Gemini
curl -s -X POST http://localhost:8080/roller-services/translate \
  -H 'Content-Type: application/json' \
  -d '{"provider":"gemini","source":"auto","target":"te","texts":["Hello world"]}' | jq .
```

Inspect same fields. Both provider requests should succeed independently; you should not need to restart the JVM to switch provider.

5. Verify cache keys include provider (browser console)
   Open browser console on a page where you used the widget and run:

```
// list cache keys
Object.keys(localStorage).filter(k => k.indexOf('roller_tx|')===0).forEach(k => console.log(k));
// inspect one entry value
console.log(localStorage.getItem('roller_tx|' + location.href + '|sarvam|te'));
```

Expect keys like: roller_tx|<page-url>|sarvam|te — provider is embedded in the key.

Optional (logs) — confirm server-side provider/cache messages

How this helped in code:

- Created direct API checks for metadata and translation endpoints.
- Validated provider switching behavior without restart.
- Verified provider-aware cache key behavior in browser localStorage.

Evaluation:

- Useful for quick regression testing after each patch.
- Practical and repeatable for team demos and grading evidence.

---

3) Runtime provider switching architecture (Sarvam vs Gemini)

Prompt used:

```
Explain how to add runtime provider switching (e.g., choose sarvam vs
gemini at request time) in this project. Sketch the high-level changes
needed in:
- TranslationService (provider factory + per-request resolve),
- REST endpoint (accept provider param),
- frontend JS (send provider).
Give only the minimal method signatures and an outline of the flow — do
not produce full class implementations.
```

Explain how to add runtime provider switching (e.g., choose sarvam vs gemini at request time) in this project. Sketch the high-level changes needed in:

- TranslationService (provider factory + per-request resolve),
- REST endpoint (accept provider param),
- frontend JS (send provider).

TranslationService.java:1-676

› Used 2 references

✓ Reviewed endpoint modifications and updated code block guidelines

High-level changes (concise)

- TranslationService
  - Ensure translate(...) overload that accepts requestedProvider calls resolveProvider(requestedProvider) — already present.
  - Ensure buildProvider(name) can construct sarvam/gemini on demand and providerPool.computeIfAbsent(...) is used (already implemented). No further change unless you need env/.env resolution in buildProvider inputs.
- REST endpoint
  - Accept a provider parameter in both GET (metadata) and POST (translate) JSON payload / query.
  - Forward the provider string unchanged to TranslationService.translate(..., provider).
  - Return provider-specific supported-languages in metadata.
- Frontend JS
  - Add a provider dropdown in the toolbar.
  - Include provider in the translate POST body and in local cache keys.
  - Make the dropdown update UI badge and save provider in localStorage.

Minimal example edits

- Servlet: accept provider and forward to service

```
// ...existing code...
protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOExceptio
    // ...existing code parsing JSON body into a Map 'payload'...
    String provider = null;
    if (payload.containsKey("provider")) {
        provider = ((String) payload.get("provider")).trim();
    }
    // collect texts, sourceLang, targetLang from payload (existing)
    List<String> texts = /* ... */;
    String src = /* ... */;
    String tgt = /* ... */;

    // Use runtime provider (null/blank => default configured provider)
    List<String> translations = translationService.translate(texts, src, tgt, provider);

    // ...existing code to build JSON response...
}
// ...existing code...
```

- Frontend: POST payload example (translation.js already follows this pattern)

```
// include provider in request body
const payload = {
  source: sourceLang,
  target: targetLang,
  provider: (providerName || _providerName || 'sarvam').toLowerCase(),
  texts: textsArray
};
fetch('/roller-services/translate', {
  method: 'POST',
```

How this helped in code:

- Clarified separation of concerns across service, servlet, and JS layers.
- Guided per-request provider resolution and providerPool reuse.
- Kept implementation minimal without overengineering.

Evaluation:

- Strong architectural guidance with low implementation risk.
- Easy to map into existing Roller code paths.

---

4) How to check if translation API is working

Prompt used:

```
how to check api is working or not
```

how to check api is wokring or not

TranslationService.java:1-676

› Used 2 references

✓ Provided API check steps and formatted code block instructions

Steps to verify the translation API is working (quick smoke tests)

1. Ensure the app is running

```
cd /home/gaurav/se_main/project-1-team-27
mvn -pl app -DskipTests jetty:run
# or ensure your server process (jetty/java) is running
pgrep -af 'jetty|java'
```

2. Check metadata (conf/.env resolution)

```
curl -s http://localhost:8080/roller-services/translate | jq .
```

Inspect JSON:

- provider → should be "sarvam" or a concrete name (not "${env...}")
- providers → array must include "sarvam" and/or "gemini"
- languages → array of supported language codes

3. Test Sarvam translation

```
curl -s -X POST http://localhost:8080/roller-services/translate \
  -H 'Content-Type: application/json' \
  -d '{"provider":"sarvam","source":"auto","target":"te","texts":["Hello world"]}' | jq .
```

Check:

- field translations exists and is an array
- translations.length == 1
- translations[0] is not identical to the input (or inspect error field)

4. Test Gemini (runtime switch)

```
curl -s -X POST http://localhost:8080/roller-services/translate \
  -H 'Content-Type: application/json' \
  -d '{"provider":"gemini","source":"auto","target":"te","texts":["Hello world"]}' | jq .
```

Same checks as above — should succeed without restarting JVM.

5. If you see placeholder errors or HTTP failures, tail server output and get stack trace

```
# If running via mvn:jetty:run, view the console where you started mvn.
# If logs are written to a file:
tail -f target/logs/roller.log
# Or find java pid and inspect environment placeholders:
pgrep -f java | xargs -r -n1 -I{} sh -c 'tr "\0" "\n" < /proc/{}/environ | grep -E "SARVAM|TRANSLATION" ||
```

Look for messages showing the literal "${env.TRANSLATION_SARVAM_API_URL}" or Illegal character in path exceptions.

6. Verify client-side caching keys include provider (in browser console)

```
// run in page console
```

How this helped in code:

- Standardized health checks: endpoint reachability, HTTP status, JSON fields.
- Added direct provider call validation to separate app-side vs upstream failures.
- Improved troubleshooting when Sarvam/Gemini behavior differs.

Evaluation:

- Simple but effective operational checklist.
- Good for quick sanity checks before deep debugging.

Task 6

Purpose

This section records additional LLM interactions used while working on **Task 6A (Discussion Overview)** and **Task 6B (Conversation Breakdown)**.

Screenshot Evidence

1) Rule-based comment classification design (Question/Feedback/Debate/Appreciation/Neutral)

ChatGPT Auto

Share

I have blog post comments with text, timestamp, and moderation status.
I need to classify each comment into one of these types: Question, Feedback, Debate, Appreciation, Neutral.
Constraint: use only rule-based methods like keywords/regex, no AI models.
Please design a practical classification approach with priority order, sample regex hints, and expected limitations.

Below is a **practical rule-based approach** (keywords + regex only) to classify blog comments into **Question, Feedback, Debate, Appreciation, Neutral**.

1. Pre-processing Steps

Before applying rules, normalize the comment text.

Steps

1. Convert text → **lowercase**
2. Remove extra whitespace
3. Remove URLs
4. Optionally remove punctuation except `?` (important for questions)

Example:

```
Original:
"Great post! I really liked this article."

Normalized:
"great post i really liked this article"
```

Ask anything

ChatGPT can make mistakes. Check important info. See Cookie Preferences.

ChatGPT Auto

Share

2. Classification Priority Order

Because comments may match multiple categories, apply **rules in priority order**.

Priority	Type
1	Question
2	Debate
3	Feedback
4	Appreciation
5	Neutral

Reason:

- Questions are easiest to detect.
- Debate often contains argument language.
- Feedback may contain suggestions/criticism.
- Appreciation is simple praise.
- Neutral = fallback.

3. Rule Definitions

3.1 Question Detection

Goal: detect information seeking.

Regex hints

Ask anything

ChatGPT can make mistakes. Check important info. See Cookie Preferences.

ChatGPT Auto

Share

Regex hints

```
\?$
^(who|what|why|when|where|how)\b
\b(can|could|should|would|is|are|do|does|did)\b.*\?
```

Keyword hints

```
how
why
what
when
where
who
can you
does this
is it possible
any idea
```

Example

```
"How do I install this plugin?"
→ Question
```

3.2 Debate Detection

Detect disagreement, argument, or opinion conflict.

Keywords

```
i disagree
this is wrong
not correct
however
```

Ask anything

ChatGPT can make mistakes. Check important info. See Cookie Preferences.

ChatGPT Auto

Share

```
but i think
i don't agree
on the contrary
that doesn't make sense
```

Regex hints

```
\b(i\s+disagree|not\s+true|wrong|however|but\s+i\s+think)\b
```

Example

```
"I disagree with your conclusion about caching."
→ Debate
```

3.3 Feedback Detection

Constructive suggestion, improvement, or critique.

Keywords

```
suggest
recommend
improve
could be better
consider adding
issue
problem
bug
needs improvement
```

Regex hints

```
\b(suggest|recommend|improve|should add|consider adding|bug|issue|problem)\b
```

Ask anything

ChatGPT can make mistakes. Check important info. See Cookie Preferences.

ChatGPT Auto

Share

Images

Example

```
"You should add an example for beginners."
→ Feedback
```

3.4 Appreciation Detection

Positive emotional reaction.

Keywords

```
great
awesome
nice post
thanks
thank you
excellent
helpful
good job
well written
```

Regex hints

```
\b(thanks?|great|awesome|excellent|nice post|good job|helpful|well written)\b
```

Example

```
"Great article, very helpful!"
→ Appreciation
```

Ask anything

ChatGPT can make mistakes. Check important info. See Cookie Preferences.

ChatGPT Auto

Share

3.5 Neutral

If **none of the above rules match**:

```
category = Neutral
```

Example:

```
"I read this yesterday."
→ Neutral
```

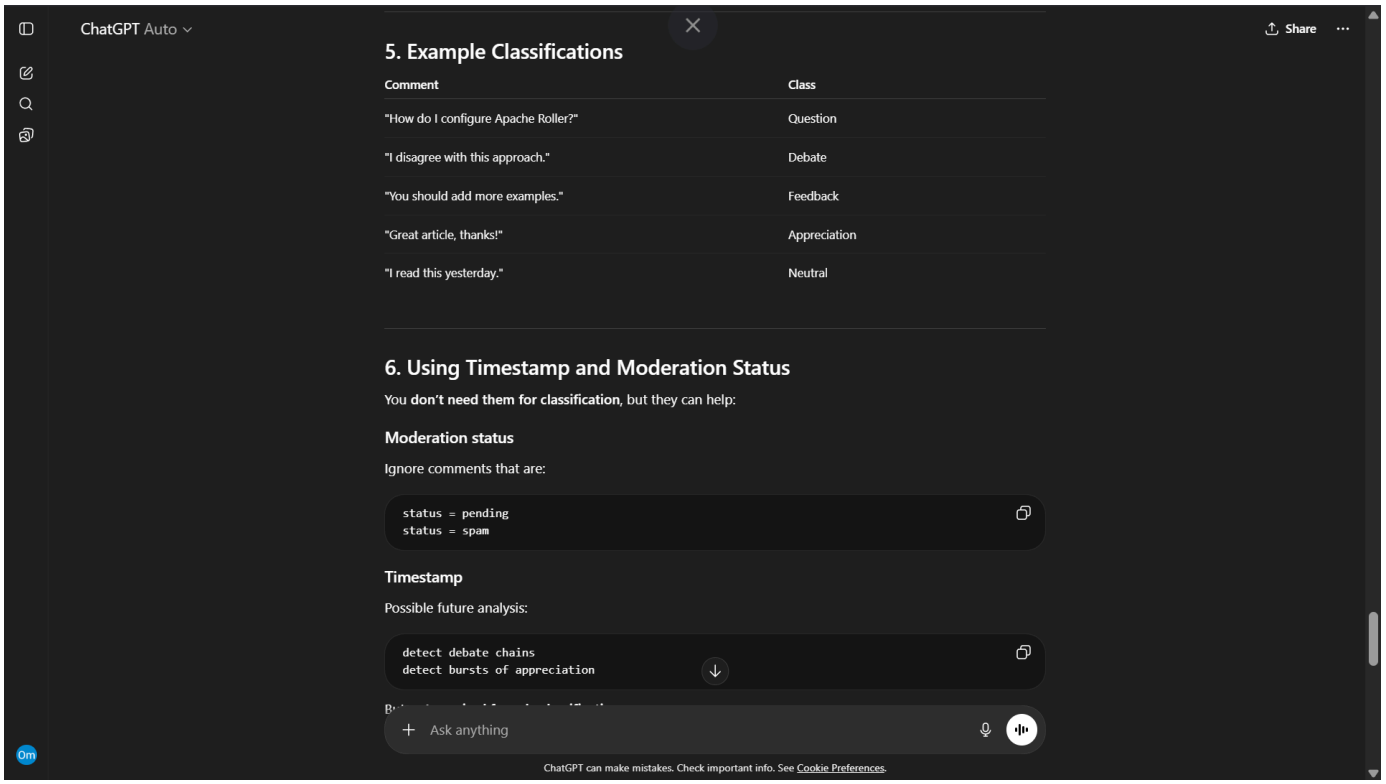
4. Example Rule-Based Logic

Pseudo-code:

```
function classify(comment):

    text = preprocess(comment)

    if matches_question(text):
        return "Question"

    if matches_debate(text):
        return "Debate"

    if matches_feedback(text):
        return "Feedback"

    if matches_appreciation(text):
        return "Appreciation"

    return "Neutral"
```

Ask anything

ChatGPT can make mistakes. Check important info. See Cookie Preferences.



Prompt used: I have blog post comments with text, timestamp, and moderation status. I need to classify each comment into one of these types: Question, Feedback, Debate, Appreciation, Neutral. Constraint: use only rule-based methods like keywords/regex, no AI models. Please design a practical classification approach with priority order, sample regex hints, and expected limitations.

How this helped in code:

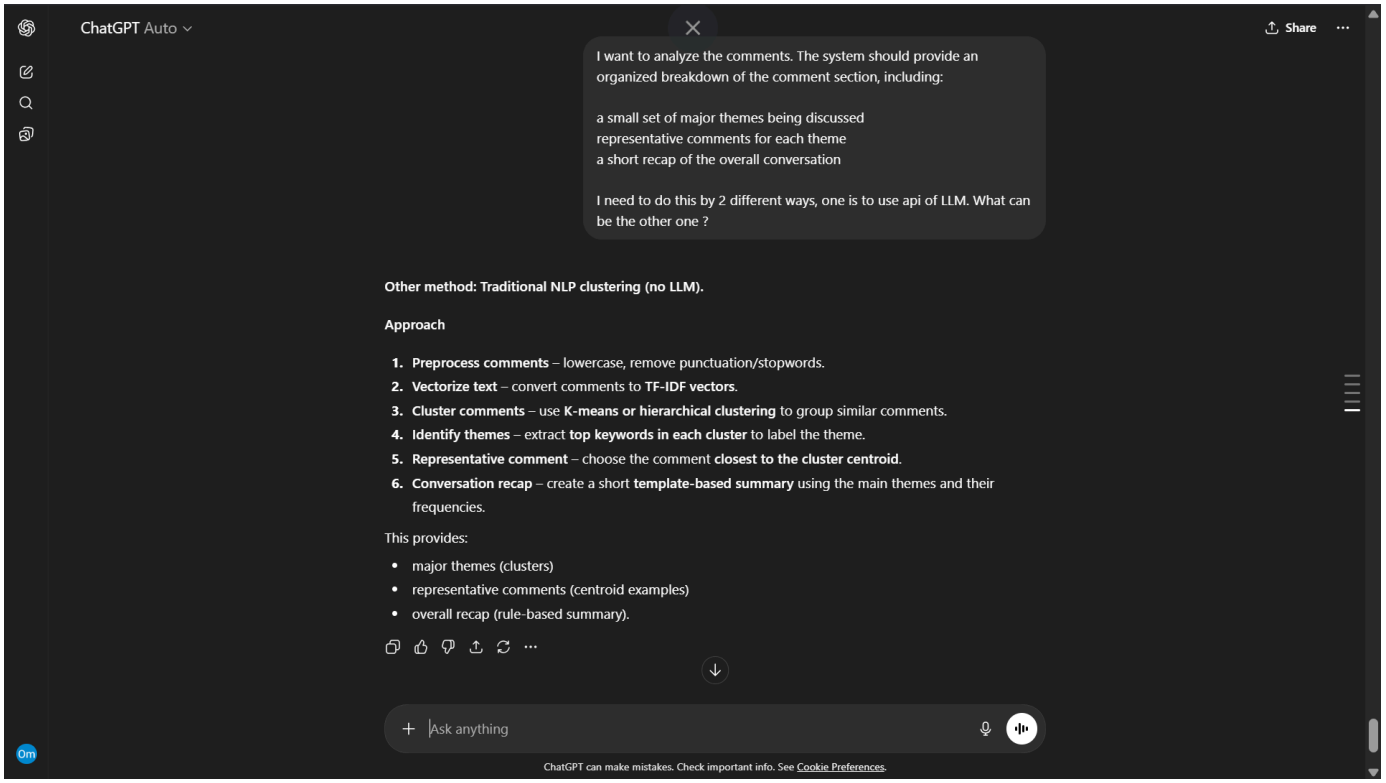
- Helped define a deterministic classification pipeline using priority-ordered rules.
- Helped shape regex and keyword sets for each category.
- Helped with practical preprocessing steps (lowercasing, whitespace cleanup, punctuation handling).

Evaluation:

- Output was aligned with Task 6A constraints (classical methods only, no LLM in runtime pipeline).
- Regex/keyword lists required manual refinement for project-specific comment patterns and false-positive reduction.
- Final category mapping and status filtering logic were manually validated during integration.

---

2) Task 6B second-method design prompt (non-LLM alternative)



Prompt used: I want to analyze the comments. The system should provide an organized breakdown of the comment section, including: a small set of major themes being discussed, representative comments for each theme, and a short recap of the overall conversation. I need to do this by 2 different ways, one is to use API of LLM. What can be the other one?

How this helped in code:

- Helped confirm a practical non-LLM alternative pipeline for Task 6B.
- Reinforced the use of classical steps: preprocessing, vectorization, clustering, theme labeling, representative selection, and template recap.
- Supported the final design decision to keep two distinct strategies (keyword/manual NLP and Gemini/LLM).

Evaluation:

- Output aligned with assignment intent of avoiding unnecessary LLM usage.
- High-level pipeline idea was useful, but concrete algorithm and integration details were implemented manually in project code.
- Final architecture used Strategy + Factory for runtime method switching and defaulted to non-LLM mode.

Final Accuracy Summary

- LLM was useful for **rule template generation, category heuristics, and identifying a second non-LLM method for Task 6B**.
- LLM output was **not used directly**; regex lists, clustering behavior, and integration details were refined manually.
- Final implementation remained compliant with Task 6 constraints:
  - lightweight classical processing for low-cost analysis
  - two distinct methods for conversation breakdown
  - optional LLM usage only when explicitly selected